

Guía práctica: Registro, Login y Gestión de Usuarios con Django

1. Introducción

En esta actividad construirás una aplicación web básica utilizando **Python y Django**.

El proyecto permitirá que una persona pueda:

- Registrarse como usuario.
- Iniciar sesión.
- Acceder a una página privada de bienvenida.
- Visualizar sus datos personales.
- Modificar algunos datos de su perfil.
- Cerrar sesión.
- Eliminar su cuenta.

Durante el desarrollo utilizarás los principales componentes de Django y trabajarás con los siguientes contenidos:

- Conexión a una base de datos a través del Framework.
- Django Admin.
- Django ORM.
- Operaciones CRUD.
- Uso de Inteligencia Artificial como apoyo para comprender y estructurar código.



2. Objetivo de aprendizaje

Al finalizar esta actividad deberías ser capaz de explicar, en términos generales, cómo funciona el siguiente flujo:

```
Usuario
  ↓
Formulario HTML
  ↓
Vista Django
  ↓
Modelo / ORM
  ↓
Base de Datos
```

También deberías ser capaz de reconocer dónde aparecen las operaciones:

```
CREATE
READ
UPDATE
DELETE
```

dentro de una aplicación Django.

3. Proyecto que construiremos

El sistema tendrá las siguientes páginas:



/registro/
/login/
/bienvenida/
/editar-perfil/
/eliminar-cuenta/

El flujo principal será:

REGISTRO
↓
LOGIN
↓
BIENVENIDA
↓
EDITAR PERFIL
↓
LOGOUT

Además, el usuario podrá eliminar su cuenta.

4. Requisitos

Antes de comenzar verifica que tengas instalado:

- Python.
- Visual Studio Code.
- Una terminal disponible.
- Acceso a Internet para instalar Django.



Para comprobar Python ejecuta:

```
python --version
```

Deberías obtener algo similar a:

```
Python 3.x.x
```

Si aparece la versión de Python, puedes continuar.

PARTE 1 — CREACIÓN DEL PROYECTO

5. Crear una carpeta

Crea una carpeta llamada:

```
sistema_usuarios
```

Puedes hacerlo desde el explorador de archivos o mediante terminal:

```
mkdir sistema_usuarios
```

Ingresa a ella:



```
cd sistema_usuarios
```

Abre esta carpeta con Visual Studio Code.

6. Crear un entorno virtual

Ejecuta:

```
python -m venv venv
```

Esto crea una carpeta llamada:

```
venv
```

¿Qué es un entorno virtual?

Un entorno virtual permite mantener las librerías utilizadas por este proyecto separadas de otros proyectos Python.

Por ejemplo, este proyecto podría utilizar una determinada versión de Django sin afectar otra aplicación.

7. Activar el entorno virtual

En Windows:

```
venv\Scripts\activate
```

Si se activó correctamente deberías observar:



(venv)

al inicio de la terminal.

Por ejemplo:

(venv) C:\proyectos\sistema_usuarios>

Punto de control

Antes de continuar verifica:

- Existe la carpeta venv.
- La terminal muestra (venv).

No continúes hasta cumplir ambos puntos.

8. Instalar Django

Ejecuta:

```
pip install django
```

Comprueba la instalación:

```
django-admin --version
```

Si aparece un número de versión, Django está instalado.

9. Crear el proyecto

Ejecuta:



```
django-admin startproject proyecto .
```

El punto . al final indica que Django debe crear el proyecto dentro de la carpeta actual.

Ahora deberías tener aproximadamente:

```
sistema_usuarios/  
├── manage.py  
├── proyecto/  
│   ├── __init__.py  
│   ├── settings.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └── wsgi.py  
└── venv/
```

10. Ejecutar Django por primera vez

Ejecuta:

```
python manage.py runserver
```

Abre en tu navegador:

<http://127.0.0.1:8000/>

Deberías observar la página inicial de Django.

Si aparece, significa que:

Python → Django → Servidor Web

están funcionando correctamente.



Detén el servidor utilizando:

CTRL + C

Pregunta de comprensión

Antes de continuar responde:

¿Para qué sirve `manage.py` dentro de un proyecto Django?

No es necesario escribir una definición perfecta. Intenta explicarlo con tus propias palabras.

PARTE 2 — CREAR NUESTRA APLICACIÓN

11. Crear la aplicación usuarios

Ejecuta:

```
python manage.py startapp usuarios
```

Ahora aparecerá:

```
usuarios/
```

con archivos como:

```
usuarios/  
|  
├── migrations/  
├── admin.py  
├── apps.py  
├── models.py  
├── tests.py  
└── views.py
```




En Django existe una diferencia entre:

Proyecto

y:

Aplicación

El proyecto contiene la configuración general.

Una aplicación representa una funcionalidad concreta.

En nuestro caso:

Proyecto:

proyecto

Aplicación:

usuarios

12. Registrar nuestra aplicación

Abre:

proyecto/settings.py

Busca:

```
INSTALLED_APPS = [
```

Agrega:

```
'usuarios',
```

Debería quedar similar a:

```
INSTALLED_APPS = [  
  
    # Aplicaciones incorporadas en Django  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    # Nuestra aplicación
```



```
        'usuarios',  
    ]
```

¿Por qué hacemos esto?

Django necesita saber qué aplicaciones forman parte de nuestro proyecto.

Crear una carpeta utilizando `startapp` no es suficiente.

También debemos registrar esa aplicación.

PARTE 3 — BASE DE DATOS

13. Revisar la conexión

Todavía dentro de:

`proyecto/settings.py`

busca:

```
DATABASES = {
```

Encontrarás:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Django utiliza por defecto:



SQLite

La base de datos será almacenada en:

`db.sqlite3`

Concepto importante

En este proyecto no realizaremos manualmente una conexión como:

```
abrir conexión  
usuario  
contraseña  
servidor  
puerto
```

Django se encargará de administrar esta conexión.

Esta es una de las ventajas de trabajar mediante un Framework.

14. Crear las tablas iniciales

Ejecuta:

```
python manage.py migrate
```

Observarás instrucciones similares a:

```
Applying auth...  
Applying admin...  
Applying sessions...
```



Ahora debería aparecer:

```
db.sqlite3
```

en la raíz del proyecto.

¿Qué ocurrió?

Django creó automáticamente las tablas que necesita para:

- Usuarios.
- Permisos.
- Sesiones.
- Administrador.
- Autenticación.

Punto de control

Verifica que exista:

```
db.sqlite3
```

Si no existe, revisa si el comando `migrate` produjo algún error.

PARTE 4 — DJANGO ADMIN

15. Crear un administrador

Ejecuta:

```
python manage.py createsuperuser
```

Django solicitará:



Username:

Email:

Password:

Puedes utilizar, por ejemplo:

Username:

admin

Utiliza una contraseña que puedas recordar durante el laboratorio.

16. Entrar a Django Admin

Ejecuta nuevamente:

```
python manage.py runserver
```

Abre:

<http://127.0.0.1:8000/admin/>

Ingresa utilizando el usuario administrador que acabas de crear.

Deberías observar una sección llamada:

Authentication and Authorization

y dentro:

Groups

Users



Ingresas a:

Users

Debería aparecer tu usuario administrador.

¿Qué estamos viendo?

Django ya tiene un modelo de usuario incorporado.

Ese modelo se llama:

User

y contiene información como:

```
username
first_name
last_name
email
password
is_staff
is_superuser
```

Pregunta

¿Por qué crees que es mejor utilizar el sistema de usuarios que Django ya incorpora en lugar de crear desde cero una tabla con usuario y contraseña?



PARTE 5 — PRIMER CONTACTO CON DJANGO ORM

17. Abrir Django Shell

Detén el servidor:

CTRL + C

Ejecuta:

```
python manage.py shell
```

Aparecerá:

```
>>>
```

Ahora estás ejecutando Python dentro del contexto de tu proyecto Django.

18. Importar User

Escribe:

```
from django.contrib.auth.models import User
```

Ahora consulta todos los usuarios:

```
User.objects.all()
```

Deberías observar algo similar a:

```
<QuerySet [<User: admin>]>
```




¿Qué significa esto?

Django está consultando la tabla de usuarios de la base de datos.

Pero nosotros no escribimos:

```
SELECT * FROM ...
```

Escribimos:

```
User.objects.all()
```

Esto es posible gracias al:

Django ORM

ORM significa:

Object Relational Mapper

Nos permite trabajar con la base de datos utilizando Python.

19. Más consultas ORM

Prueba:

```
User.objects.count()
```

Ahora:

```
User.objects.filter(is_active=True)
```

Busca tu administrador:

```
User.objects.get(username='admin')
```

Guárdalo:

```
usuario = User.objects.get(username='admin')
```

Consulta:

```
usuario.username
```

Luego:

```
usuario.email
```



Pregunta

Explica con tus palabras la diferencia entre:

`all()`

`filter()`

y:

`get()`

Cuando termines:

`exit()`



PARTE 6 — CREAR EL REGISTRO DE USUARIOS

20. Crear forms.py

Dentro de:

usuarios/

crea un archivo:

forms.py

Escribe:

```
# Importamos el sistema de formularios de Django.
from django import forms

# Django ya incorpora un formulario diseñado
# especialmente para crear usuarios.
from django.contrib.auth.forms import UserCreationForm

# Importamos el modelo de usuario incorporado
# por Django.
from django.contrib.auth.models import User

# Creamos nuestro formulario de registro.
#
# Heredamos de UserCreationForm para aprovechar
# las validaciones de usuarios y contraseñas
# que Django ya incorpora.
class RegistroUsuarioForm(UserCreationForm):

    # Agregamos el correo electrónico porque
```



```
# queremos solicitarlo obligatoriamente.
email = forms.EmailField(
    required=True,
    label='Correo electrónico'
)

class Meta:

    # Indicamos que este formulario
    # trabaja con el modelo User.
    model = User

    # Definimos los campos que aparecerán
    # en nuestro formulario.
    fields = [
        'username',
        'first_name',
        'last_name',
        'email',
        'password1',
        'password2',
    ]
```

21. Entender el formulario

Observa:

```
class RegistroUsuarioForm(UserCreationForm):
```

Nuestro formulario **hereda** funcionalidades del formulario de creación de usuarios de Django.

Eso significa que no necesitamos crear manualmente toda la lógica relacionada con contraseñas.

Además:

```
model = User
```

indica que el formulario se encuentra relacionado con el modelo de usuarios.

Y:

```
fields = [...]
```

determina qué campos aparecerán.

Pregunta

¿Por qué aparecen:

```
password1
```

```
password2
```

en vez de solamente un campo password?

PARTE 7 — CREAR LA VISTA DE REGISTRO

22. Modificar views.py

Abre:

`usuarios/views.py`

Escribe:

```
# render permite cargar un archivo HTML.
#
# redirect permite enviar al usuario
# hacia otra página.
from django.shortcuts import render, redirect

# Importamos nuestro formulario.
from .forms import RegistroUsuarioForm

# Vista responsable del registro.
def registro(request):

    # Comprobamos si el navegador está
    # enviando información mediante POST.
    if request.method == 'POST':

        # Creamos un formulario utilizando
        # la información recibida.
        form = RegistroUsuarioForm(
            request.POST
        )

        # Verificamos que los datos sean válidos.
        if form.is_valid():

            # Guardamos el usuario.
            #
```



```
# Django se encargará de almacenarlo
# en la base de datos.
form.save()

# Después del registro enviamos
# al usuario a la página de login.
return redirect('login')

else:

    # Si solamente estamos entrando
    # a la página, creamos un formulario vacío.
    form = RegistroUsuarioForm()

# Mostramos el archivo HTML.
return render(
    request,
    'usuarios/registro.html',
    {
        'form': form
    }
)
```

23. Entender request

Todas las vistas reciben:

`request`

`request` contiene información relacionada con la petición realizada por el navegador.

Por ejemplo:

`request.method`



puede indicar:

GET

o:

POST

En términos sencillos:

GET

normalmente significa:

Quiero obtener una página.

Mientras que:

POST

normalmente significa:

Estoy enviando información.

PARTE 8 — TEMPLATES

24. Crear carpetas

Dentro de:

usuarios/

crea:

templates/



Dentro:

usuarios/

La estructura será:

```
usuarios/  
|  
├── templates/  
│   └── usuarios/  
|  
├── admin.py  
├── apps.py  
├── forms.py  
├── models.py  
└── views.py
```

25. Crear registro.html

Dentro de:

usuarios/templates/usuarios/

crea:

registro.html

Escribe:

```
<!DOCTYPE html>
```

```
<html lang="es">
```



```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Registro</title>
```

```
</head>
```

```
<body>
```

```
    <h1>
```

```
        Crear una cuenta
```

```
    </h1>
```

```
    <!--
```

```
        El formulario enviará sus datos  
        utilizando el método POST.
```

```
-->
```

```
    <form method="POST">
```

```
        <!--
```

```
            Protección de seguridad  
            utilizada por Django.
```

```
-->
```

```
        {% csrf_token %}
```

```
    <!--
```

```
        Django generará los campos  
        de nuestro formulario.
```

```
-->
```

```
    {{ form.as_p }}
```

```
    <button type="submit">
```

```
        Registrarse
```

```
    </button>
```



```
</form>
```

```
<hr>
```

```
<p>
```

```
    ¿Ya tienes una cuenta?
```

```
    <a href="{% url 'login' %}">
```

```
        Iniciar sesión
```

```
    </a>
```

```
</p>
```

```
</body>
```

```
</html>
```

26. ¿Qué significa {{ }}?

Cuando escribimos:

```
{{ form.as_p }}
```

estamos mostrando una variable enviada desde Django.

En cambio:

```
{% csrf_token %}
```

representa una instrucción de Django Template Language.



Por lo tanto:

```
{{  }}
```

normalmente muestra información.

Mientras que:

```
{% %}
```

ejecuta instrucciones del template.

PARTE 9 — CONFIGURAR URLs

27. Crear usuarios/urls.py

Dentro de:

usuarios/

crea:

urls.py

Escribe:

```
# Importamos path.  
from django.urls import path
```

```
# Importamos nuestras vistas.  
from . import views
```

```
urlpatterns = [
```

```
# Dirección:
#
# /registro/
path(
    'registro/',
    views.registro,
    name='registro'
),
]
```

28. Conectar las URLs

Abre:

proyecto/urls.py

Modifica el archivo:

```
from django.contrib import admin

# include permite incorporar
# las URLs de otra aplicación.
from django.urls import path, include

urlpatterns = [

    # Django Admin
    path(
        'admin/',
        admin.site.urls
    ),

    # URLs de nuestra aplicación usuarios.
```



```
        path(
            '',
            include('usuarios.urls')
        ),
    ]
```

Ahora nuestro proyecto conoce las URLs creadas dentro de:

`usuarios/urls.py`

PARTE 10 — CREAR LOGIN

29. Utilizar LoginView

Django ya incorpora una vista para iniciar sesión.

En:

`usuarios/urls.py`

agrega:

```
from django.contrib.auth.views import LoginView
```

El archivo debería quedar:

```
from django.urls import path
```

```
from django.contrib.auth.views import LoginView
```

```
from . import views
```

```
urlpatterns = [
    # Registro
```

```
path(
    'registro/',
    views.registro,
    name='registro'
),

# Login
path(
    'login/',
    LoginView.as_view(
        template_name='usuarios/login.html'
    ),
    name='login'
),
]
```

30. Crear login.html

Crea:

`usuarios/templates/usuarios/login.html`

Escribe:

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Iniciar sesión</title>
```

```
</head>
```




```
<body>
```

```
    <h1>
```

```
        Iniciar sesión
```

```
    </h1>
```

```
    <form method="POST">
```

```
        {% csrf_token %}
```

```
        <!-- Django crea automáticamente
```

```
            los campos de usuario y contraseña. -->
```

```
        {{ form.as_p }}
```

```
        <button type="submit">
```

```
            Ingresar
```

```
        </button>
```

```
    </form>
```

```
    <hr>
```

```
    <p>
```

```
        ¿No tienes una cuenta?
```

```
        <a href="{% url 'registro' %}">
```

```
            Registrarse
```


</p>

</body>

</html>

PARTE 11 — PÁGINA DE BIENVENIDA

31. Crear una vista privada

En:

`usuarios/views.py`

agrega esta importación:

```
from django.contrib.auth.decorators import login_required
```

Después agrega:

```
# login_required significa que solamente
# usuarios autenticados pueden acceder.
@login_required
def bienvenida(request):

    return render(
        request,
        'usuarios/bienvenida.html'
    )
```

32. Crear la URL

En:

`usuarios/urls.py`

agrega:

```
path(
    'bienvenida/',
    views.bienvenida,
    name='bienvenida'
),
```

33. Crear bienvenida.html

Crea:

`usuarios/templates/usuarios/bienvenida.html`

Escribe:

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```



```
<meta charset="UTF-8">

<title>Bienvenida</title>

</head>

<body>

  <!-- request.user representa
        al usuario autenticado. -->

  <h1>

    Bienvenido {{ request.user.username }}

  </h1>

  <p>

    Has iniciado sesión correctamente.

  </p>

  <hr>

  <h2>
    Mis datos
  </h2>

  <p>

    <strong>Usuario:</strong>

    {{ request.user.username }}

  </p>
```

```
<p>

    <strong>Nombre:</strong>

    {{ request.user.first_name }}

</p>

<p>

    <strong>Apellido:</strong>

    {{ request.user.last_name }}

</p>

<p>

    <strong>Correo:</strong>

    {{ request.user.email }}

</p>

</body>

</html>
```

34. Configurar la redirección

Abre:



proyecto/settings.py

Al final agrega:

```
# Después de iniciar sesión correctamente,  
# Django enviará al usuario a esta página.  
LOGIN_REDIRECT_URL = 'bienvenida'
```

```
# Si una página requiere autenticación  
# y el usuario no está conectado,  
# Django lo enviará al login.  
LOGIN_URL = 'login'
```

35. Probar el sistema

Ejecuta:

```
python manage.py runserver
```

Ingresa a:

<http://127.0.0.1:8000/registro/>

Crea un usuario.

Por ejemplo:

Usuario:
alumno1

Nombre:
Ana

Apellido:
Pérez



Correo:

ana@example.com

Utiliza una contraseña válida.

Después del registro deberías llegar a:

`/login/`

Inicia sesión.

Si todo está correcto llegarás a:

`/bienvenida/`

Punto de control

Antes de avanzar verifica:

- Puedes registrar un usuario.
- El usuario aparece en Django Admin.
- Puedes iniciar sesión.
- Puedes acceder a `/bienvenida/`.
- El nombre del usuario aparece en pantalla.

PARTE 12 — RELACIÓN CON CREATE Y READ

Ya hemos implementado dos operaciones CRUD.



CREATE

Cuando hacemos:

```
form.save()
```

durante el registro:

CREATE

Estamos creando un nuevo usuario.

READ

Cuando utilizamos:

```
{{ request.user.username }}
```

estamos leyendo información del usuario.

Por lo tanto:

CREATE → Registro

READ → Página de bienvenida

PARTE 13 — EDITAR EL PERFIL

36. Crear formulario de edición

Abre:

```
usuarios/forms.py
```




Al final agrega:

```
# Formulario para editar
# datos del usuario.
class EditarUsuarioForm(forms.ModelForm):

    class Meta:

        # Trabajamos con User.
        model = User

        # Solamente permitiremos modificar
        # estos campos.
        fields = [
            'first_name',
            'last_name',
            'email',
        ]
```

37. Importarlo en views.py

Actualmente tendrás:

```
from .forms import RegistroUsuarioForm
```

Cámbialo por:

```
from .forms import (
    RegistroUsuarioForm,
    EditarUsuarioForm,
```

)

38. Crear la vista editar_perfil

Agrega:

```
@login_required
def editar_perfil(request):
    # Si recibimos información del formulario...
    if request.method == 'POST':

        # request.POST contiene los nuevos datos.
        #
        # instance=request.user indica que
        # modificaremos al usuario actual.
        form = EditarUsuarioForm(
            request.POST,
            instance=request.user
        )

        if form.is_valid():

            # Guardamos los cambios.
            form.save()

            # Regresamos a bienvenida.
            return redirect('bienvenida')

    else:

        # Cuando solamente abrimos la página,
        # mostramos los datos actuales.
        form = EditarUsuarioForm(
            instance=request.user
        )
```

```
return render(  
    request,  
    'usuarios/editar_perfil.html',  
    {  
        'form': form  
    }  
)
```

39. Entender instance

Observa:

```
instance=request.user
```

Sin instance, un `ModelForm` podría interpretar que queremos crear un nuevo objeto.

Cuando indicamos:

```
instance=request.user
```

estamos diciendo:

Trabaja utilizando este usuario existente.

Por eso esta operación corresponde a:

UPDATE

40. Crear editar_perfil.html

Crea:

```
usuarios/templates/usuarios/editar_perfil.html  
<!DOCTYPE html>
```



```
<html lang="es">

<head>

    <meta charset="UTF-8">

    <title>Editar perfil</title>

</head>

<body>

    <h1>
        Editar perfil
    </h1>

    <form method="POST">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">
            Guardar cambios
        </button>

    </form>

    <br>
```



```
<a href="{% url 'bienvenida' %}">
```

Volver

```
</a>
```

```
</body>
```

```
</html>
```

41. Agregar URL editar

En:

`usuarios/urls.py`

agrega:

```
path(
    'editar-perfil/',
    views.editar_perfil,
    name='editar_perfil'
),
```

42. Agregar acceso desde bienvenida

En:



bienvenida.html

agrega:

```
<a href="{% url 'editar_perfil' %}">
```

Editar mis datos

```
</a>
```

Prueba modificar:

- Nombre.
- Apellido.
- Correo.

Luego verifica que la página de bienvenida muestre los cambios.

PARTE 14 — ELIMINAR CUENTA

43. Crear vista eliminar_cuenta

Agrega en:

usuarios/views.py

@login_required

def eliminar_cuenta(request):

Por seguridad solamente eliminamos

si la petición utiliza POST.

if request.method == 'POST':

Obtenemos al usuario autenticado.

usuario = request.user

Eliminamos el registro.

usuario.delete()

```
# Volvemos al login.
return redirect('login')

# Si todavía no confirmó,
# mostramos una página de confirmación.
return render(
    request,
    'usuarios/eliminar_cuenta.html'
)
```

44. Crear eliminar_cuenta.html

Crea:

```
usuarios/templates/usuarios/eliminar_cuenta.html
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Eliminar cuenta</title>
```

```
</head>
```

```
<body>
```

```
    <h1>
```

```
        Eliminar cuenta
```

```
    </h1>
```



<p>
¿Está seguro que desea eliminar su cuenta?

</p>

<p>
Esta acción no se puede deshacer.

</p>

<form method="POST">

{% csrf_token %}

<button type="submit">

Sí, eliminar mi cuenta

</button>

</form>

Cancelar

</body>

</html>

45. Crear URL eliminar

Agrega:

```
path(  
    'eliminar-cuenta/',  
    views.eliminar_cuenta,  
    name='eliminar_cuenta'  
),
```

46. Agregar acceso

Dentro de:

bienvenida.html

agrega:

```
<a href="{% url 'eliminar_cuenta' %}">
```

Eliminar mi cuenta

```
</a>
```



Ahora tenemos:

DELETE

PARTE 15 — CERRAR SESIÓN

47. Agregar LogoutView

Modifica la importación de:

LoginView

para que quede:

```
from django.contrib.auth.views import (  
    LoginView,  
    LogoutView,  
)
```

Agrega:

```
path(  
    'logout/',  
    LogoutView.as_view(),  
    name='logout'  
)
```

48. Configurar la redirección del logout

En:

proyecto/settings.py



agrega:

```
LOGOUT_REDIRECT_URL = 'login'
```

49. Agregar botón cerrar sesión

Dentro de:

bienvenida.html

agrega:

```
<form
    method="POST"
    action="{% url 'logout' %}"
>
```

```
    {% csrf_token %}
```

```
    <button type="submit">
```

```
        Cerrar sesión
```

```
    </button>
```

```
</form>
```

Prueba cerrar sesión.

Deberías regresar a:

/login/

PARTE 16 — CRUD COMPLETO

En este punto has implementado:

Operación	Acción
CREATE	Registrar usuario
READ	Mostrar datos
UPDATE	Editar perfil
DELETE	Eliminar cuenta

Esto corresponde a:

CRUD

que significa:

Create

Read

Update

Delete

PARTE 17 — PROFUNDIZAR EN DJANGO ORM

50. Abrir Django Shell

Ejecuta:

```
python manage.py shell
```



Importa:

```
from django.contrib.auth.models import User
```

51. Obtener todos los usuarios

```
User.objects.all()
```

52. Contarlos

```
User.objects.count()
```

53. Filtrar activos

```
User.objects.filter(  
    is_active=True  
)
```

54. Filtrar administradores

```
User.objects.filter(  
    is_superuser=True  
)
```

55. Buscar por username

```
User.objects.get(  
    username='admin'  
)
```

56. Buscar correos que contengan texto

```
User.objects.filter(  
    email__contains='gmail'  
)
```

El operador:

`__contains`

permite buscar registros cuyo campo contiene un determinado texto.

57. Ver el SQL generado

Escribe:

```
consulta = User.objects.filter(  
    is_active=True  
)
```

Después:

```
print(  
    consulta.query  
)
```



Django mostrará una consulta SQL.

Esto demuestra el flujo:

Código Python



Django ORM



SQL



Base de Datos

PARTE 18 — USO DE INTELIGENCIA ARTIFICIAL

La Inteligencia Artificial puede ayudarte durante el desarrollo.

Sin embargo, su objetivo en esta actividad será:

comprender

revisar

explicar

detectar errores

proponer mejoras

y no simplemente generar todo el proyecto.

58. Actividad con IA N°1

Selecciona la función:



```
def editar_perfil(request):
```

y pregúntale a Copilot o ChatGPT:

Estoy aprendiendo Django.

Explicame paso a paso esta función.

No quiero que la reemplaces ni que escribas una versión diferente.

Quiero entender:

1. Qué es request.
2. Qué significa request.method.
3. Qué contiene request.POST.
4. Qué significa instance=request.user.
5. Qué hace form.is_valid().
6. Qué hace form.save().
7. Qué hace redirect().

Utiliza lenguaje sencillo para alguien que está comenzando con Django.

Lee la respuesta.

Luego escribe con tus propias palabras:

¿Qué función cumple `instance=request.user`?

59. Actividad con IA N°2

Entrega a la IA tu:

`forms.py`



Utiliza:

Estoy aprendiendo Django.

Revisa este archivo `forms.py`.

No escribas una solución completa todavía.

Indícame:

1. Qué está correctamente implementado.
2. Qué validaciones realiza Django automáticamente.
3. Qué posibles mejoras observas.
4. Qué parte debería entender mejor.
5. Propón solamente una mejora sencilla y explícame por qué sería útil.

Código:

[PEGAR AQUÍ TU FORMS.PY]

No implementes automáticamente todas las recomendaciones.

Primero determina si comprendes el cambio sugerido.

PARTE 19 — DESAFÍOS

Realiza los siguientes ejercicios sin copiar inmediatamente una solución generada por IA.

Desafío 1

En la página de bienvenida muestra:

Fecha de creación de la cuenta



Pista:

El modelo User contiene:

`date_joined`

Desafío 2

Mostrar un mensaje diferente dependiendo de si el usuario es administrador.

Investiga:

`request.user.is_superuser`

Resultado esperado:

Si es administrador:

Este usuario es administrador.

Si no:

Este usuario es un usuario normal.

Desafío 3

Utilizando Django ORM encuentra:

Todos los usuarios cuyo nombre comience con A.

Investiga el operador:

`__startswith`



Desafío 4

Utilizando Django Admin modifica manualmente el correo electrónico de uno de tus usuarios.

Después verifica desde la página de bienvenida que el cambio sea visible.

PARTE 20 — PREGUNTAS DE COMPRENSIÓN

Responde utilizando tus propias palabras.

Pregunta 1

¿Qué diferencia existe entre:

Proyecto Django

y:

Aplicación Django



Pregunta 2

¿Qué función cumple:

```
python manage.py migrate
```

?

Pregunta 3

¿Qué es Django ORM?

Pregunta 4

¿Cuál es la diferencia entre:

```
User.objects.all()
```

y:

```
User.objects.filter(...)
```

?

Pregunta 5

¿Qué función cumple:

```
@login_required
```

?

Pregunta 6

¿Por qué Django utiliza:

```
{% csrf_token %}
```

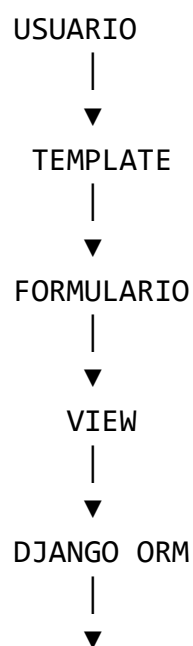
en formularios POST?

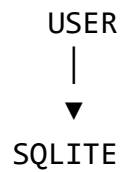
Pregunta 7

Explica dónde aparecen las operaciones CRUD dentro de nuestro proyecto.

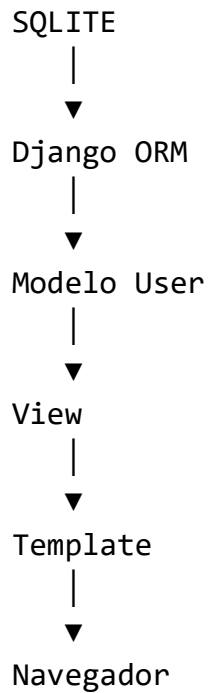
PARTE 21 — MAPA DEL PROYECTO

Al finalizar deberías comprender aproximadamente este flujo:





Cuando Django obtiene información:



PARTE 22 — ESTRUCTURA FINAL

Tu proyecto debería terminar aproximadamente así:

```
sistema_usuarios/  
|  
|— db.sqlite3  
|— manage.py  
|  
|— proyecto/  
|   |— __init__.py  
|   |— settings.py  
|   |— urls.py
```

```
|
|   ├── asgi.py
|   └── wsgi.py
|
|── usuarios/
|   ├── migrations/
|   |
|   ├── templates/
|   |   └── usuarios/
|   |       ├── registro.html
|   |       ├── login.html
|   |       ├── bienvenida.html
|   |       ├── editar_perfil.html
|   |       └── eliminar_cuenta.html
|   |
|   ├── admin.py
|   ├── apps.py
|   ├── forms.py
|   ├── models.py
|   ├── tests.py
|   ├── urls.py
|   └── views.py
|
└── venv/
```

PARTE 23 — CHECKLIST FINAL

Marca cada elemento solamente cuando hayas comprobado que funciona.

- [] Creé un entorno virtual.
- [] Instalé Django.
- [] Creé el proyecto.
- [] Creé la aplicación usuarios.
- [] Registré la aplicación en settings.py.



- [] Ejecuté las migraciones.
- [] Creé un superusuario.
- [] Ingresé a Django Admin.
- [] Consulté usuarios mediante Django ORM.
- [] Construí el formulario de registro.
- [] Registré un usuario.
- [] Implementé el login.
- [] Implementé una página protegida.
- [] Mostré los datos del usuario.
- [] Implementé edición de perfil.
- [] Implementé eliminación de cuenta.
- [] Implementé logout.
- [] Identifiqué las cuatro operaciones CRUD.
- [] Revisé una parte del código utilizando IA.
- [] Puedo explicar aproximadamente el flujo completo de la aplicación.

Resultado esperado

Al terminar la actividad deberías tener una aplicación funcional donde:



Una persona se registra
↓
Sus datos llegan al servidor
↓
Django valida el formulario
↓
Django ORM guarda el usuario
↓
SQLite almacena la información
↓
El usuario inicia sesión
↓
Django crea una sesión
↓
El usuario accede a una página privada
↓
Puede consultar y modificar sus datos

Más importante que terminar rápidamente es comprender **qué responsabilidad tiene cada componente**.

Si algo deja de funcionar durante el laboratorio, revisa primero:

1. El mensaje de error.
2. El archivo indicado por Django.
3. La línea indicada por Django.
4. Si guardaste los archivos.
5. Si escribiste correctamente los nombres de las URLs.
6. Si el servidor sigue ejecutándose.
7. Si realizaste las migraciones correspondientes.

El objetivo no es solamente que la aplicación funcione, sino que al finalizar seas capaz de explicar cómo Django conecta formularios, vistas, modelos, ORM y base de datos para construir una aplicación web.

